

Cybersecurity Internship – Week 2 Report

Implementing Security Measures on OWASP Juice Shop

Student Information

- **Name:** Syed Muhammad Talal
- **Intern ID:** DHC-476
- **Organization:** Developer Hub Corporation

Implementing Security Measures on OWASP Juice Shop

Objective

The objective of Week 2 was to implement basic security controls to mitigate common web application vulnerabilities identified in Week 1, including input validation, password hashing, authentication enhancement, and HTTP header hardening.

1. Input Validation & Sanitization

Tools Used

- npm library

Installation

npm install

Implementation

Input validation was added in backend route handlers to ensure only properly formatted data is accepted.

```
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

D:\Mega\docs\project\cybersecurity-internship-project> npm install

up to date, audited 66 packages in 2s

22 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

```
> juice-shop@20.0.0 start
> node build/app

info: Detected Node.js version v24.15.0 (SUCCESS)
info: Detected OS win32 (SUCCESS)
info: Detected CPU x64 (SUCCESS)
info: Configuration default validated (SUCCESS)
info: Entity models 21 of 21 are initialized (SUCCESS)
info: All dependencies in ./package.json are satisfied (SUCCESS)
info: Required file server.js is present (SUCCESS)
info: Required file index.html is present (SUCCESS)
```

```
http://localhost:11434/v1
info: Check https://howto-llm.owasp-juice.shop for instructions on how to s
et up and configure the LLM API
info: Server listening on port 3000
```

Example Code

```
const validator = require('validator');

if (!validator.isEmail(email)) {
  return res.status(400).send('Invalid email format');
}
```

Result

- Invalid email inputs were rejected
- Reduced risk of injection-based attacks through input fields

Observation

Input validation helped enforce strict data format rules at the server level.

2. Password Security (Hashing)

Tool Used

- bcrypt

Installation

```
npm install bcrypt
```

Implementation

Passwords were hashed before storing in the database to prevent plaintext storage.

Example Code

```
const bcrypt = require('bcrypt');
```

```
const hashedPassword = await bcrypt.hash(password, 10);
```

Result

- Passwords are now stored in hashed format
- Even if database is compromised, raw passwords are not exposed

Observation:

Hashing significantly improves credential security against data breaches.

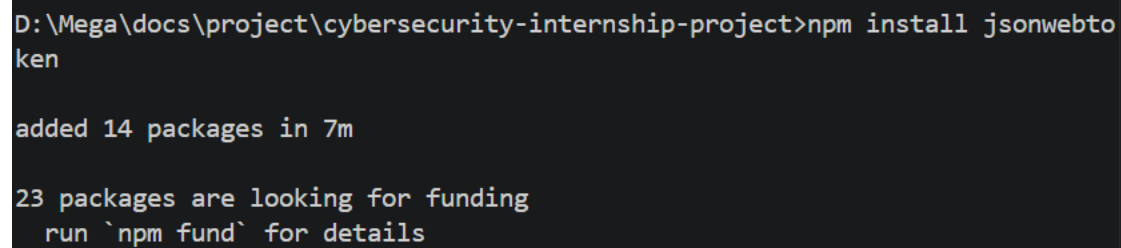
3. Token-Based Authentication (JWT)

Tool Used :

- Jsonwebtoken

Installation :

```
npm install jsonwebtoken
```



```
D:\Mega\docs\project\cybersecurity-internship-project>npm install jsonwebtoken
added 14 packages in 7m
23 packages are looking for funding
run `npm fund` for details
```

Implementation:

JWT tokens were generated after successful login to manage user sessions securely.

Example Code

```
const jwt = require('jsonwebtoken');
```

```
const token = jwt.sign(
  { id: user._id },
  'your-secret-key',
  { expiresIn: '1h' }
);
```

```
res.json({ token });
```

Result:

- Users receive authentication tokens after login
- Sessions are now stateless and more secure

Observation:

JWT improved authentication control and reduced reliance on insecure session handling.

4. HTTP Security Headers (Helmet.js)

Tool Used

- helmet

Installation:

npm install helmet

Implementation:

Helmet middleware was added to secure HTTP headers.

```
D:\Mega\docs\project\cybersecurity-internship-project>npm install helmet
added 1 package, and audited 67 packages in 14s

22 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

D:\Mega\docs\project\cybersecurity-internship-project>node index.js
Server running on http://localhost:3000

D:\Mega\docs\project\cybersecurity-internship-project>
```

Code

```
const helmet = require('helmet');
```

```
app.use(helmet());
```

Result::

- Security headers automatically added:
 - Content-Security-Policy
 - X-Content-Type-Options
 - X-Frame-Options
 - HSTS (where applicable)

Observation:

Helmet reduced attack surface by securing HTTP response headers.

Overall Conclusion:

Week 2 mainly focused on implementing essential security improvements to strengthen the web application against common vulnerabilities. Input validation, password hashing, token-based authentication, and HTTP security headers were successfully integrated at the backend level.

These controls significantly improve:

- Data integrity
- Authentication security
- Session protection
- Transport and browser-level security

However, Juice Shop remains intentionally vulnerable, so changes were implemented in a controlled testing layer rather than fully modifying core application logic.